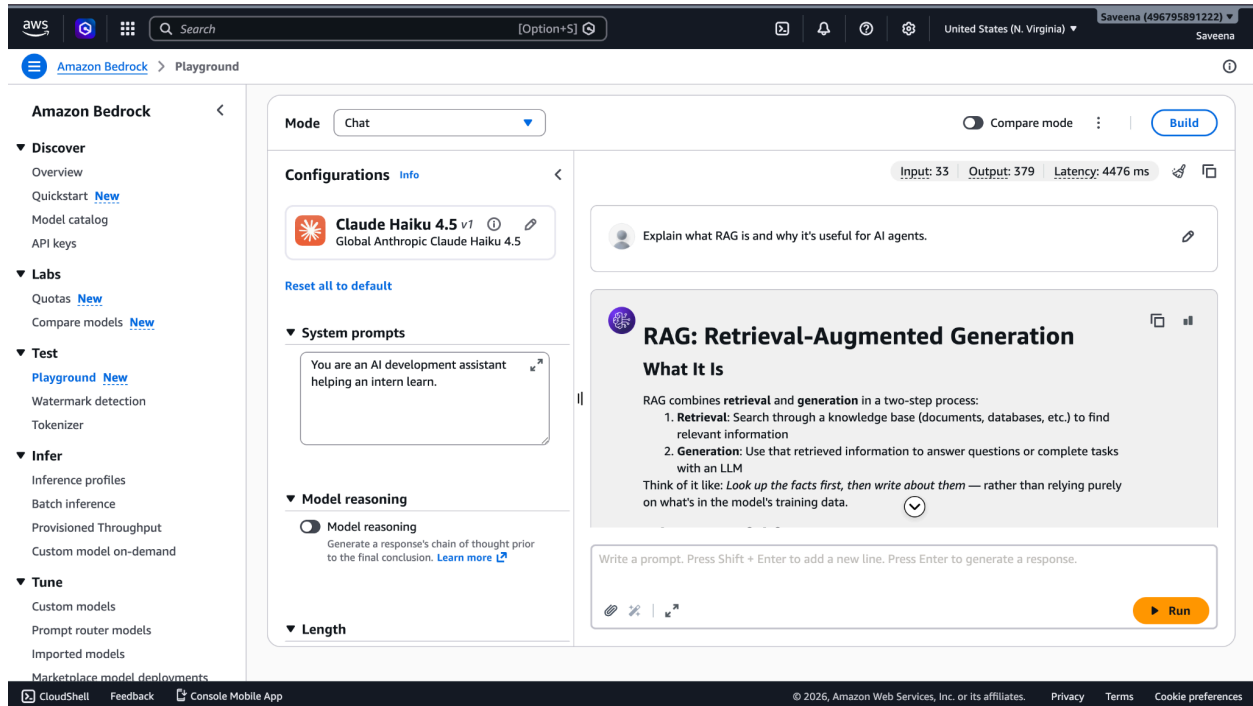


Invoke a model via the Amazon Bedrock Playground and replicate the call using boto3 in Python

Invoking Model:



Copy of Result:

User: Explain what RAG is and why it's useful for AI agents.

Model: # RAG: Retrieval-Augmented Generation

What It Is

RAG combines **retrieval** and **generation** in a two-step process:

1. **Retrieval**: Search through a knowledge base (documents, databases, etc.) to find relevant information
2. **Generation**: Use that retrieved information to answer questions or complete tasks with an LLM

Think of it like: *Look up the facts first, then write about them* — rather than relying purely on what's in the model's training data.

Why It's Useful for AI Agents

1. **Access to Current Information**

- LLMs have training cutoffs; RAG lets agents use real-time data
- Example: An agent can answer questions about today's stock prices or recent news

2. **Domain-Specific Knowledge**

- Agents can access proprietary documents, internal databases, or specialized content
- Example: Customer support agents equipped with product manuals and FAQs

3. **Accuracy & Grounding**

- Reduces "hallucinations" by basing answers on actual retrieved content
- Easier to verify sources and audit decisions

4. **Cost Efficiency**

- No need to fine-tune models on new data
- Just update your knowledge base

5. **Scalability**

- Add new information without retraining
- Works with large document sets

Simple Example

****Without RAG:**** Agent tries to answer from memory alone → might be outdated or wrong

****With RAG:**** Agent searches docs → finds "Policy updated March 2024" → generates accurate response

This makes agents more reliable, current, and trustworthy for production use.

Replicate the call using boto3 in Python

```
(venv) saveenaboga@Saveenas-MacBook-Air task2-ai-agent %  
(venv) saveenaboga@Saveenas-MacBook-Air task2-ai-agent % python3 ~/bedrock_call.py  
# RAG (Retrieval-Augmented Generation) Explained  
  
## What is RAG?  
  
RAG is a technique that combines two key components:  
  
1. Retrieval - Finding relevant information from an external knowledge source (documents, databases, web, etc.)  
2. Generation - Using that retrieved information to produce better AI responses  
  
[The workflow is simple:  
- User asks a question → System retrieves relevant documents → LLM reads those docs + question → Generates informed answer  
]  
  
## Why RAG is Useful for AI Agents  
[  
### 1. Access to Current Information  
- LLMs have knowledge cutoff dates (training data ends at a point in time)  
- RAG lets agents fetch real-time data, recent news, or updated databases  
- Example: An agent can answer "What's the current stock price?" by retrieving live data  
  
### 2. Domain-Specific Knowledge  
- RAG lets you ground agents in proprietary data (company docs, manuals, customer records)  
- Reduces hallucinations by giving the agent facts to cite  
- Example: A support agent retrieving from your help documentation  
  
### 3. Reduced Hallucinations  
- Instead of the LLM *guessing*, it has actual source material  
- Agent can cite where information came from  
- Builds trust in responses  
  
### 4. Scalability Without Retraining  
- Add new documents without retraining the entire model  
- Cost-effective compared to fine-tuning  
- Easy to update knowledge bases  
  
### 5. Explainability  
- You can see *which documents* influenced the answer  
- Easier to debug if responses are wrong  
- Important for compliance and transparency  
]  
  
## Simple Example  
...  
Without RAG: "Tell me about Company X's Q3 earnings"  
→ LLM might hallucinate outdated numbers  
  
With RAG: "Tell me about Company X's Q3 earnings"  
→ System retrieves Q3 earnings report  
→ LLM answers based on actual document  
→ Much more accurate!  
...  
  
RAG essentially gives your AI agent a "reference library" to consult before answering.  
(venv) saveenaboga@Saveenas-MacBook-Air task2-ai-agent % █
```